EXPRESS.JS
Building REST APIs and Advanced Web Applications

Assignment Article / Report

GitHub Repository

https://github.com/khushipg04-tech/version-control-testing-project

A comprehensive guide to Express.js covering setup, routing, middleware, REST APIs, error handling, security, and performance optimization.

1. Introduction to Express.js

Express.js is a lightweight and flexible web framework for Node.js. It simplifies the creation of web servers, web applications, and REST APIs. Express provides routing, middleware support, request and response handling, and easy integration with databases and npm packages.

Since Express runs on Node.js, applications can use JavaScript on the server side and benefit from Node.js's asynchronous, non-blocking I/O model.

2. Overview and Key Features

• Simple framework for Node.js web development.

• Routing for GET, POST, PUT, PATCH, and DELETE requests.

• Middleware support for logging, authentication, parsing, and error handling.

• Easy creation of RESTful APIs.

• Support for static files and template engines.

• Large ecosystem through npm.

• Flexible integration with databases and external services.

3. Setting Up an Express Application

Install Node.js and npm, create a project folder, initialize it, and install Express.

```
mkdir express-app
cd express-app
npm init -y
npm install express
```

Create app.js:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello from Express.js!");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Run the server with:

```
node app.js
```

4. Routing in Express.js

Routing determines how an application responds to a request for a particular path and HTTP method.

```
app.get("/users", (req, res) => {
  res.send("List of users");
});

app.post("/users", (req, res) => {
  res.send("Create a user");
});

app.put("/users/:id", (req, res) => {
  res.send("Update user " + req.params.id);
});

app.delete("/users/:id", (req, res) => {
  res.send("Delete user " + req.params.id);
});
```

Route parameters use colon syntax, for example /users/:id. The value is available through req.params.id.

5. Middleware

Middleware functions run during the request-response cycle. They can inspect or modify request and response objects, perform a task, and call next() to pass control to the next middleware or route handler.

```
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

app.use(express.json());
app.use(express.static("public"));
```

6. Building REST APIs with Express

A REST API exposes resources through URLs and uses HTTP methods for operations. GET retrieves data, POST creates resources, PUT or PATCH updates resources, and DELETE removes resources.

```
const express = require("express");
const app = express();
app.use(express.json());

let books = [
  { id: 1, title: "The Alchemist", author: "Paulo Coelho" },
  { id: 2, title: "Clean Code", author: "Robert C. Martin" }
];

app.get("/api/books", (req, res) => {
  res.status(200).json(books);
});

app.post("/api/books", (req, res) => {
  const book = {
    id: books.length + 1,
    title: req.body.title,
    author: req.body.author
```

```
  };
  books.push(book);
  res.status(201).json(book);
});

app.put("/api/books/:id", (req, res) => {
  const book = books.find(b => b.id == req.params.id);
  if (!book) return res.status(404).json({message:"Book not found"});
  book.title = req.body.title ?? book.title;
  book.author = req.body.author ?? book.author;
  res.json(book);
});

app.delete("/api/books/:id", (req, res) => {
  books = books.filter(b => b.id != req.params.id);
  res.status(204).send();
});

app.listen(3000);
```

7. HTTP Status Codes

Common REST API status codes include 200 OK for success, 201 Created after creation, 204 No Content for a successful operation without a response body, 400 Bad Request for invalid input, 401 Unauthorized for missing or invalid authentication, 403 Forbidden for insufficient permission, 404 Not Found when a resource is missing, and 500 Internal Server Error for unexpected server failures.

8. Request and Response Handling

Express exposes request information through req. Query parameters use req.query, route parameters use req.params, and parsed JSON request data uses req.body.

```
app.get("/search", (req, res) => {
  res.json({ searchTerm: req.query.q });
});

app.post("/users", (req, res) => {
  res.status(201).json(req.body);
});
```

9. Error Handling

Centralized error-handling middleware makes an application easier to maintain. Error middleware has four parameters: err, req, res, and next.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: "Internal server error" });
});
```

Asynchronous operations should be handled so that rejected promises and thrown errors reach the application's error handler.

10. Advanced Express Techniques

10.1 Router Modules

Express Router helps divide a large application into smaller route modules.

```
const express = require("express");
const router = express.Router();

router.get("/", (req, res) => {
  res.json({ message: "All users" });
});

router.get("/:id", (req, res) => {
  res.json({ id: req.params.id });
});
```

```
module.exports = router;

// app.js
app.use("/api/users", userRouter);
```

10.2 Authentication and Authorization

Authentication verifies identity, while authorization determines permissions. Express applications can use sessions, JWT-based tokens, or external identity providers. Secrets and credentials should not be hard-coded or committed to a public repository.

10.3 Input Validation

Validate request bodies, parameters, and query strings before processing them. Validation prevents invalid data and reduces unexpected input problems.

10.4 Security

Important practices include HTTPS in production, secure authentication, input validation, appropriate CORS configuration, safe error messages, and security-related HTTP headers. Helmet is commonly used to help set security headers, while rate limiting can reduce abusive request traffic.

11. Performance Optimization

• Use asynchronous, non-blocking I/O operations.

• Cache frequently requested data when appropriate.

• Use database indexes for common queries.

• Avoid unnecessary middleware and repeated computation.

• Use pagination for large result sets.

• Use clustering or multiple processes when appropriate.

• Monitor response times, errors, CPU, memory, and database performance.

```
// Simple caching concept
const cache = new Map();

app.get("/api/data", (req, res) => {
  if (cache.has("data")) return res.json(cache.get("data"));
  const data = { message: "Example data" };
  cache.set("data", data);
  res.json(data);
});
```

12. REST API Best Practices

• Use resource-oriented URLs such as /api/books and /api/books/:id.

• Use HTTP methods according to their intended meaning.

• Return meaningful status codes.

• Validate incoming data.

• Use consistent JSON response structures.

• Implement centralized error handling.

• Protect sensitive routes with authentication and authorization.

• Document endpoints and request/response formats.

• Use pagination, filtering, and sorting for large collections.

13. Example Project Structure

```
express-app/
■■■ app.js
■■■ package.json
■■■ routes/
■   ■■■ books.js
■   ■■■ users.js
■■■ middleware/
■   ■■■ auth.js
■   ■■■ errorHandler.js
■■■ controllers/
■   ■■■ booksController.js
■   ■■■ usersController.js
■■■ public/
```

Separating routes, middleware, and controllers makes larger Express applications easier to understand, test, and maintain.

14. Advantages and Limitations

Advantages include a small learning curve, flexible architecture, a large npm ecosystem, strong suitability for REST APIs, and easy integration with databases and services.

Express is intentionally minimal, so developers must select and configure many additional components. Poor middleware ordering, missing validation, inefficient database queries, and inadequate error handling can reduce application quality.

15. Testing and Deployment

Express APIs can be tested using browser requests for simple GET endpoints and API clients for POST, PUT, PATCH, and DELETE operations. Automated tests can verify routes, middleware, validation, authentication, and error handling.

Production deployments should use environment variables for configuration, secure connections, logging, monitoring, and an appropriate process or deployment strategy.

16. Learning Outcomes

• Explain Express.js and its relationship with Node.js.

• Create and configure an Express application.

• Define routes using HTTP methods.

• Use built-in and custom middleware.

• Build REST APIs using CRUD operations.

• Handle errors and validate requests.

• Apply security and performance optimization techniques.

• Organize an Express project into maintainable modules.

17. Conclusion

Express.js provides a simple and flexible foundation for building web applications and REST APIs with Node.js. Its routing and middleware systems support modular request processing, while the npm ecosystem provides tools for authentication, validation, databases, security, and other requirements. Good API design, error handling, validation, security, caching, and monitoring help create maintainable and scalable Express

applications.

18. Submission Details

Title: Express.js: Building REST APIs and Advanced Web Applications

Link: https://github.com/khushipg04-tech/version-control-testing-project

PDF: Express_JS_REST_API_Assignment.pdf